Rethinking Spreadsheets

Gavin Lowe

March 25, 2026

1 Introduction

General rationale for the project. ...

The language is influenced by both Scala and Haskell. In particular, the language allows type checking of the script.

A script contains a combination of declarations (of values and functions), and commands (e.g. to write a value into a cell). The script is executed in order.

2 A simple example script

Figure 1 gives a simple (somewhat contrived) example script, intended to introduce the language. The script expects to find integers in an initial segment of column A. It calculates their factorials, and writes them in column B. It also prints the total of those factorials below.

The characters "\\" Turn the rest of the line into a comment.

Line 2 defines two values In and Out, representing the input and output columns. A column value can be defined by preceding the corresponding letter with "#", so, for example, "#A" represents column A. Likewise a row value can be defined by preceding the row number with "#", for example "#0". It is a good idea to give names to relevant rows and columns, in the same way that it is a good idea to give names to relevant constants in standard programming languages.

Line 5 defines the factorial function. Definitions of functions are introduced using the keyword **def**. Each parameter of a function is given a type. The return type of a function should be given when the function is recursive—which is the case for all three functions in this script—or when a reference is made to the function before its definition (e.g. when two functions are defined

```
1  // The input and output columns.
2  val In = #A; val Out = #B
3
4  // The factorial of n.
5  def fact(n: Int): Int = if(n <= 1) 1 else n*fact(n-1)
6
7  // The sum of xs.
8  def sum(xs: List[Int]): Int = if(xs == []) 0 else head(xs) + sum(tail(xs))
9
10 // Row for the first empty cell in column c from row r.
11 def firstEmptyInCol(c: Column, r: Row): Row =
12   Cell(c,r) match{ case Empty => r; case n: Int => firstEmptyInCol(c, r+1) }
13
14 // Row for the first empty cell in In
15 val firstEmpty = firstEmptyInCol(In, #0)
16
17 // Write factorials of entries in In into Out.
18 for(r <- #0 until firstEmpty) Cell(Out,r) = fact(Cell(In,r))
19
20 // Write the total.
21 Cell(In, firstEmpty+1) = "Total:"
22 Cell(Out, firstEmpty+1) = sum([Cell(Out,r): Int | r <- #0 until firstEmpty])
```

Figure 1: A simple example script.

by mutual recursion). The definition of fact is a straightforward recursion, whose meaning should be obvious.

The sum function at line 8 calculates the sum of a list of Ints. If A is a type, then List[A] is the type of lists that contain values of type A. The value [] is the empty list. More generally, a list can be defined by writing its elements between square brackets, separated by commas, for example "[2, 4, 6, 8]". The function head returns the first element of a nonempty list; and the function tail returns the list containing all except the first element of a nonempty list. Again, the definition is a straightforward recursion.

The expression Cell(c,r) represents the value in the spreadsheet cell in column c and row r. In the case where c and r are column and row literals, this can be written more concisely. For example, #B3 is shorthand for Cell(#B,#3).

The function firstEmptyInCol(c, r), at line 11, finds the first empty cell in column c, from row r onwards, and returns the relevant row. Column is the type of columns, and Row is the type of rows. When a cell is read, the script

should define what type or types the cell may read. The firstEmptyInCol function requires the cell read to be either empty or an integer. This is checked using pattern matching on the value, using syntax similar to that in Scala. Each case in a pattern matching expression is of the form **case** *pat* => *e*, where *pat* is a pattern, and *e* is an expression giving the result when that pattern is matched. The pattern Empty matches an empty cell. Hence, if Cell(c,r) is empty, firstEmptyInCol returns r. The pattern n: Int matches an integer, and binds the name n to that integer in the subsequent expression. Hence, if Cell(r,r) contains an integer, the firstEmptyInCol function recurses with firstEmptyInCol(c, r+1), where r+1 represents the row after r; thus, the function simply continues searching from the next row. We could have written the pattern as "_: Int" in this case, where "_" represents a wildcard pattern, because the subsequent expression does not use the value of the integer. If the value read from the cell does not match any of the patterns—in this case, if it is not empty or an integer—the pattern matching fails, and the spreadsheet application gives an error message.

Line 15 sets firstEmpty to be the first row in column In that contains an empty cell.

A cell can be written using a command of the form "Cell(c,r) = *e*", where *e* defines the value written. For example, line 21 writes the string "Total:" into the cell below the first empty cell in column In.

Line 18 writes into column Out the factorials of values in corresponding cells in column In. It does this using a **for** loop. A **for** loop of the form **for**(x <− xs) *block* executes *block* for each value of x from the list xs. Here *block* is a block containing declarations and commands. The expression #0 **until** firstEmpty represents the list of rows from 0, inclusive, until firstEmpty, exclusive, i.e. the rows that were previously found to contain integers in column In. For each such row r, line 18 writes the corresponding factorial into column Out.

Finally, line 22 calculates the sum of the factorials, and writes it into the appropriate cell. The expression in square brackets on the left is a list comprehension, using Haskell-style notation. It forms the list of values in row r of column Out, for each value of r in #0 **until** firstEmpty, i.e. the factorials written at line 18. The subexpression "Cell(Out, r): Int" tells the spreadsheet program that the cell should contain an integer; the typechecker makes use of this information.

3 Using the spreadsheet application

The spreadsheet application can be run by running Scala on the class `spreadsheet.SpreadsheetApp`, providing the name of the script to use. The application uses the Scala Swing library, so that needs to be on the classpath. I use the following command (except all on one line):

```
scala -cp .:/home/gavin/Scala/lib/scala-swing_2.13-3.0.0.jar
  spreadsheet.SpreadsheetApp factorials.dir
```

Optionally, the name of a CSV file can also be provided, giving values to load into the spreadsheet.

Values can be entered into the spreadsheet in the normal way. The script is run automatically each time the user enters a value. Figure 2 gives a screenshot of the window after the user has input values into the first three cells in column A. Cells with a purple background are user inputs; cells with a green background contain values written by the script, with the currently selected cell in a darker shade.

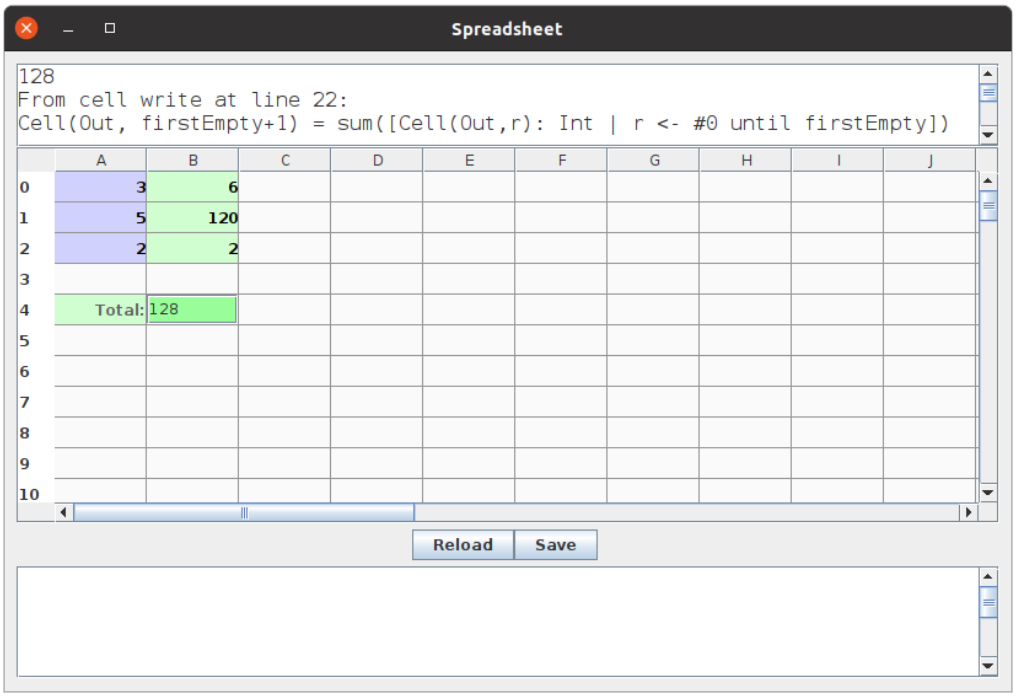

Figure 2: The spreadsheet application window.

The top panel in the window, known as the *selection panel*. gives information about the selected cell. Here it shows that the cell contains the

value 128, that the value came from a cell write at line 22; it also gives the corresponding code.

As with any programming language, errors can occur, which can be detected either when the script is type checked, or when the script is executed.

As an example of a type error, suppose we change line 5 of the script to the following:

```
def fact(n: Int): Int = if(n <= 1.0) 1 else n*fact(n-1)
```

This is a type error, because it compares the Int n with the floating point number 1.0. Figure 3 shows what happens when the script is re-loaded (via the "Reload" button). The bottom panel, known as the *error panel*, gives information about each error. Here it indicates that the script expected an Int but found a Float at line 5, and that the erroneous term was 1.0; it also gives enclosing code at various levels.

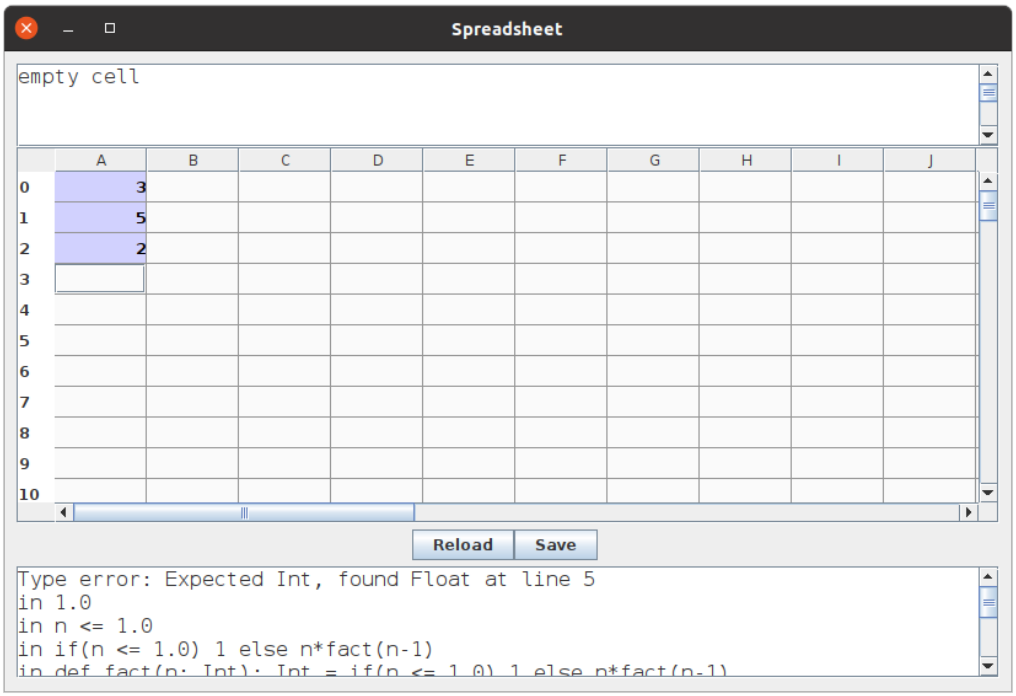

Figure 3: The spreadsheet application indicating a type error.

Execution-time errors include errors common in other programming languages, such as division by zero, or taking the head of an empty list.

A non-standard form of error occurs when the script reads a value from a cell that is not of the expected type. For example, suppose we enter the string three into cell #A0, the error panel displays the following error.

```
Cannot match String in cell #A0 at line 12
in "Cell(c,r) match{ case Empty => r; case _: Int => firstEmptyInCol(c, r+1) }"
in "firstEmptyInCol(In, #0)"
in "val firstEmpty = firstEmptyInCol(In, #0)"
```

This shows that the pattern matching at line 12 fails when given a string.

Similar errors can occur outside pattern matching. Suppose we change the definition of firstEmptyInCol as follows.

```
def firstEmptyInCol(c: Column, r: Row): Row =
  Cell(c,r) match{ case Empty => r; case _ => firstEmptyInCol(c, r+1) }
```

The pattern "_" matches any value, so the pattern matching will succeed. However, now other errors occur, as illustrated in Figure 4, where cells with a pink background represent error. The error panel indicates that the cell #A0 read at line 18 found a String when an Int was expected. The same information is included in the selection panel, since cell #B0 is selected. The bottom of the error panel also explains an error a type error corresponding to the write of cell #B4 (this requires scrolling to read fully).

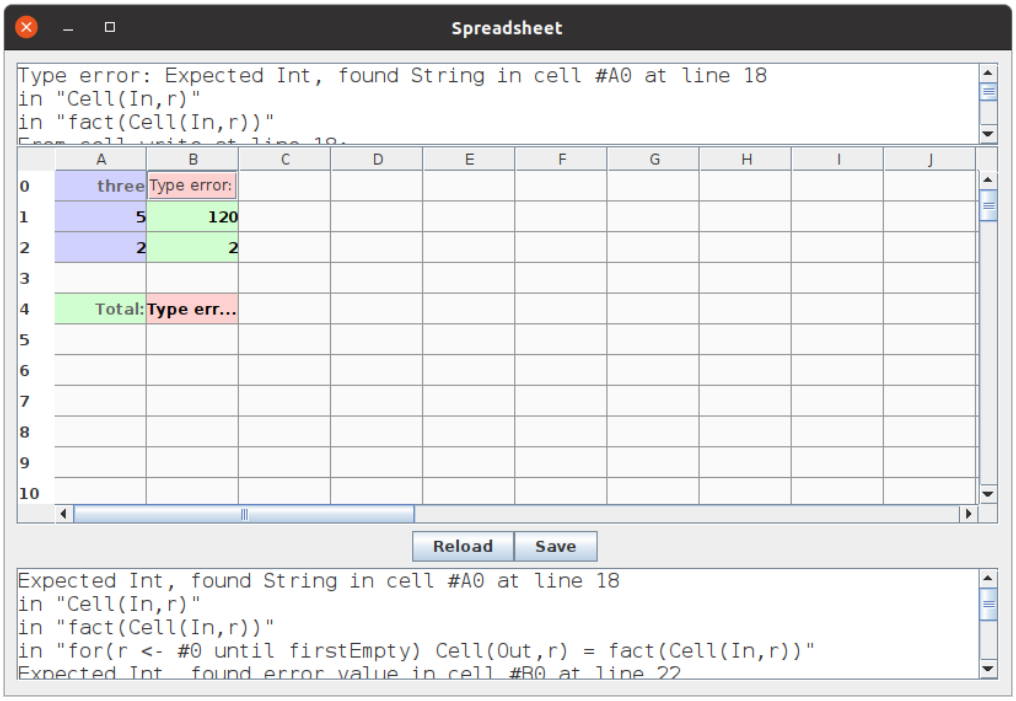

Figure 4: The spreadsheet application indicating a cell read error.

The application partitions cells into *input cells*, where the user has provided input, *output cells*, written to by the script, and *empty cells*. Any

attempt to write to an output cell is considered an error and blocked: this prevents an erroneous script from overwriting user data. Likewise, an attempt to write to an output cell for a second time is considered an error.

The "Save" button on the spreadsheet can be used to store the contents of input cells in a CSV file.

4 Script language

4.1 Types

The script language contains the standard types Int of (32-bit) integers, Float of (32-bit) floating point numbers, Boolean of booleans (with values true and false), and String of strings.

These four types can be entered in cells of the spreadsheet. The application decides their type based on how they are entered, mostly in the obvious way; for example, if a value looks like an integer, it is given the type Int. In the case that a String is required that looks like a value of another type, it can be entered inside quotation marks (e.g. "3"); in such cases, a literal quotation can be entered as \" (e.g. "He said "Hello"").

The types Row and Column represent rows and columns, respectively.

If T is a type, then List[T] represents the type of lists whose elements are from T.

4.2 Expressions

4.2.1 Standard expressions

The script language contains the normal arithmetic operators, +, −, ∗ and /, over Int and Float (over Int, / performs rounding). Note that Int and Float are distinct types, so can't (currently) be combined using these operators. The language also contains the normal equality (==), inequality (!=), and order (<, <=, >, >=) operators.

The language includes the normal boolean binary operators, && and ||. Boolean negation is written using the function not.

Conditional expressions are written using C/Java/Scala-style syntax:

if(⟨test⟩) ⟨expression⟩ **else** ⟨expression⟩

The two branches must have the same type, and this is the overall type of the conditional expression.

Functional applications can be written using parentheses, in the normal way, as in Figure 1. However, as in Haskell, the parentheses can be omitted

for a function of one argument, where the actual parameter does not logically need parentheses: for example, f 3, or g #A3. Function application associates to the left, and binds tighter than infix operators; for example, f g true + h #A represents ((f(g))(true)) + (h(#A)).

String literals are written inside double quotation marks, as in most other languages, e.g. "Hello". Special characters can be written using \, for example, "He said \"Hello\".\n".

A *block expression* consists of a sequence of declarations, followed by an expression, separated by semicolons or newlines, and enclosed in braces, for example

```
{ def square(x: Int) = x*x; val y2 = square y; val z2 = square z; y2+z2 }
```

The declarations are evaluated in order; then the value of the final expression is the value of the block expression.

4.2.2 List expressions

List literals are written in square brackets, for example [1,2,3] or []. All the elements of the list must be of the same type. With the empty list, it is sometimes necessary to give the type, to help the type checker, for example []: List[Int].

List concatenation is written using ::, pronounced "cons". This operator takes an element on the left and a list on the right, and returns their concatenation; for example 3 :: [4,5] = [3,4,5]. The function head returns the first element of a non-empty list; for example head [3,4,5] = 3. The function tail returns the list containing all except the first element of a non-empty list; for example tail [3,4,5] = [4,5]. The function isEmpty tests whether a list is empty.

Lists of consecutive Ints can be constructed using the infix operators **to** and **until**. The list m **to** n produces the list from m to n inclusive; for example, 3 to 5 = [3,4,5]. The list m **until** n is similar, but does not include n; for example, 3 until 5 = [3,4]. These operators can also be applied to row and column values; see below.

List comprehensions provide a way to define a list, in a similar style to mathematical set comprehensions. The expression [e | x <− xs] generates the list containing the value of e for each x in xs. For example,

```
[x*x | x <- [3,4,5]]  =  [9,16,25].
```

A term of the form x <− xs is called a *generator*. A list comprehension may have multiple generators, separated by commas; an earlier generator is

iterated over more slowly than a later generator. For example,

```
[x*y | x <- [2,3], y <- [4,5]]  =  [8,10,12,15].
```

A list comprehension may also have one or more boolean filters; a term is produced only for values of the variables that satisfy the filter. For example,

```
[x*y | x <- [2,3], y <- [4,5], x+y != 7]  =  [8,15].
```

4.2.3 Row, column, and cell expressions

Row and column literals are written using #, e.g. #A, #B, etc., and #0, #1, etc. A row or column value can have an Int added to it or subtracted from it, with the obvious effect. For example, #C + 2 = #E, and #C − 2 = #A. Arithmetic underflow, e.g. from #C − 3, gives an error.

Lists of row and column values can be formed using the **to** and **until** operators. For example #B **to** #D = [#B,#C,#D], and #2 **until** #4 = [#2,#3].

If c and r are a Row and Column, respectively, then Cell(c,r) is a *cell expression*: it represents the value in the corresponding cell of the spreadsheet. This can be abbreviated in the case of row and column literals; for example #D3 is shorthand for Cell(#D, #3).

The type checker needs to be able to deduce a type, or a set of possible types, for each cell expression. When the script is executed, the application checks if the corresponding cell in the spreadsheet has the expected type, and signals an error otherwise.

One way the type deduction can be made is if a function is applied to the cell expression: this implies that its type must match the parameter type for the function. For example, in line 18 of Figure 1, the expression fact(Cell(In,r)) implies that the cell expression must have type Int, since fact takes a parameter of type Int.

Alternatively, an explicit type can be provided for a cell expression. For example, #D3: Int gives the type Int to the cell expression.

Finally, a **match** expression allows the cell to have one of several different types, and for the value of the expression to depend on that type. An example was given on line 12 of Figure 1. In general, a **match** expression takes the form

⟨cell expression⟩ **match**{ ⟨cases⟩ }

where ⟨cases⟩ is a sequence of cases, separated by semicolons or newlines. Each case is of the form

case ⟨pattern⟩ => ⟨expression⟩

Each pattern takes one of the following forms.

- Empty, which matches an empty cell;
- ⟨name⟩: ⟨type⟩, where ⟨type⟩ is one of Int, Float, String, or Boolean; this matches a value of the given type, and binds the name to that value in the subsequent expression;
- _: ⟨type⟩, which is similar to the previous form, except no name is bound;
- _, which matches any type.

In a **match** expression, the cases are tried in the order given; the first case that matches is selected, and the corresponding expression is evaluated. If no case matches, an error is signalled. Within a **match** expression, all the expressions on the right-hand side of cases must have the same type.

4.3 Declarations

A *value declaration* is written using the keyword **val**. Figure 1 contains several examples.

A *function declaration* is written using the keyword **def**. Again, Figure 1 contains several examples. Each parameter of the function should be given a type. A return type must be given for a recursive function, or if a forward reference is made to the function. For example, the return type for f is necessary in the following:

```
val y = f(3); def f(x: Int): Int = x+1
```

However, no return type is necessary in the following:

```
def f(x: Int) = x+1; val y = f(3)
```

val
def
return types needed.
multiple argument lists
polymorphism. Eq type class.
overloading
Top-level or local.

4.4 Directives